Distributed Processing of ATLAS Open Data Using Docker and Kubernetes

Abstract

A sequential ATLAS Open Data analysis was restructured as a distributed system using message-based execution and containerisation. Individual data files were processed in parallel using RabbitMQ to queue tasks between multiple workers, before being recombined to reproduce identical outputs. Kubernetes deployed the workflow, enabling improved scalability and workload management, highlighting the suitability of a cloud-based system.

Introduction

The dependency on distributed computing systems has drastically increased for large-scale scientific experiments in order in order to process and analyse large volumes of data. Data sizes in fields such as particle physics, astronomy and genomics routinely exceed what can be handled by a single machine, resulting in modern scientific computing relying on cloud-based platforms and automated orchestration tools which can distribute workloads across multiple processing nodes, improving the overall performance, reproducibility and fault tolerance.[1]

The ATLAS experiment at the Large Hadron Collider is an example of modern scientific computing, producing data at an effective rate of 1PB/s whilst in operation, which is far too large of a production of data to be processed or stored by a single machine. The data heavily filtered and compressed daily by a local data centre, however the annual dataset, spanning up to tens of PB, depends on globally distributed computing resources, with thousands of jobs running concurrently across large compute clusters, proving the necessity of parallelism.[2]

The ATLAS Open Data represents structured collisions of reconstructed event records, made public for analysis. A GitHub repository code to process this data (linked in Ref. 3) imports large collections of structured ROOT files and produces outputs in the form of histograms and summary statistics to describe the data.[3] This current analysis processes each ROOT file sequentially within a single execution thread, resulting in a long runtime which scales linearly with the number of input files. Due to the independence of the processing of each file, the workload can be distributed in parallel across multiple workers in a cloud-based system, allowing simultaneous analysis, significantly improving scalability.

This workload therefore motivates a message-driven execution model, where individual processing tasks are placed into a queue and dynamically assigned to available compute resources by a message broker. Containerisation enables the analysis code and its dependencies to be packaged into portable execution units, ensuring consistent runtime environments. These containers can then be deployed, monitored, and scaled across a distributed system using an orchestration platform, providing a structured approach to executing scientific workflows on cloud infrastructure.

Methodology

A GitHub repository was initialised to manage version control and provide a reproducible structure for the cloud-based workflow development.[4] The base code, HZZ.py, was first refactored into hzz_batch.py, where the direct execution loop over all input files was removed, instead reorganising the computation so that each ROOT file could be processed independently.

File processing was separated into a callable unit, making it possible to treat each file as an independent task rather than part of a single monolithic script. This restructuring was essential for enabling parallel execution, since it decoupled the workload into smaller work items that could be scheduled across multiple workers without introducing shared state or race conditions.

Next, the code was converted into a message-driven framework using RabbitMQ, where a producer, workers and a combiner were implemented, with the producer having extracted relevant file paths and submitted them to a queue. Multiple workers then process the filenames and individually execute the analysis, sending the resulting partial histogram to a second queue where a combiner process aggregated all the partial outputs into the final complete histogram and summary statistics. This approach ensured that communication between components occurred only through the message broker, avoiding direct coupling between services and enabling horizontal scalability of workers without code modification.

Docker was used to implement containerisation which ensured consistent execution environments across services and to package all dependencies required. A Dockerfile was created to define the base runtime environment, which installed all requirements necessary. This system was firstly deployed using Docker Compose, with the creation of a docker-compose.yml file where each major process was defined as a service: RabbitMQ as the message broker, one producer service, multiple workers, and a single combiner service. Volume mounts were used to persist output files generated by the combiner container, allowing data to be retrieved from outside the container environment.

Once this method was working, the system was moved to implementing Kubernetes in order to introduce orchestration and resource management at cluster level as Docker Compose orchestrated containers on a single node, whereas Kubernetes introduced cluster-level scheduling, pod abstraction and persistent volumes. Kubernetes deployments were created for the always-on components, including RabbitMQ and the worker pool, allowing worker replicas to be scaled independently of other services. The producer and combiner were deployed as Kubernetes jobs rather than services or deployments in order to ensure that they only executed once per run and terminated upon completion automatically. Persistent storage was implemented through a PersistentVolumeClaim (PVC), enabling the combiner outputs to be preserved even after the container exited. An auxiliary helper pod was then used to mount the same volume and copy the results to local storage, providing a simple mechanism for extracting output files from the cluster.

A shell script was written to deploy all Kubernetes resources, restart completed jobs, wait for completion of producer and combiner jobs and extract results automatically, reducing human input to a single ./run_k8s.sh command. Figure 1 below shows the full workflow in this Kubernetes implementation.

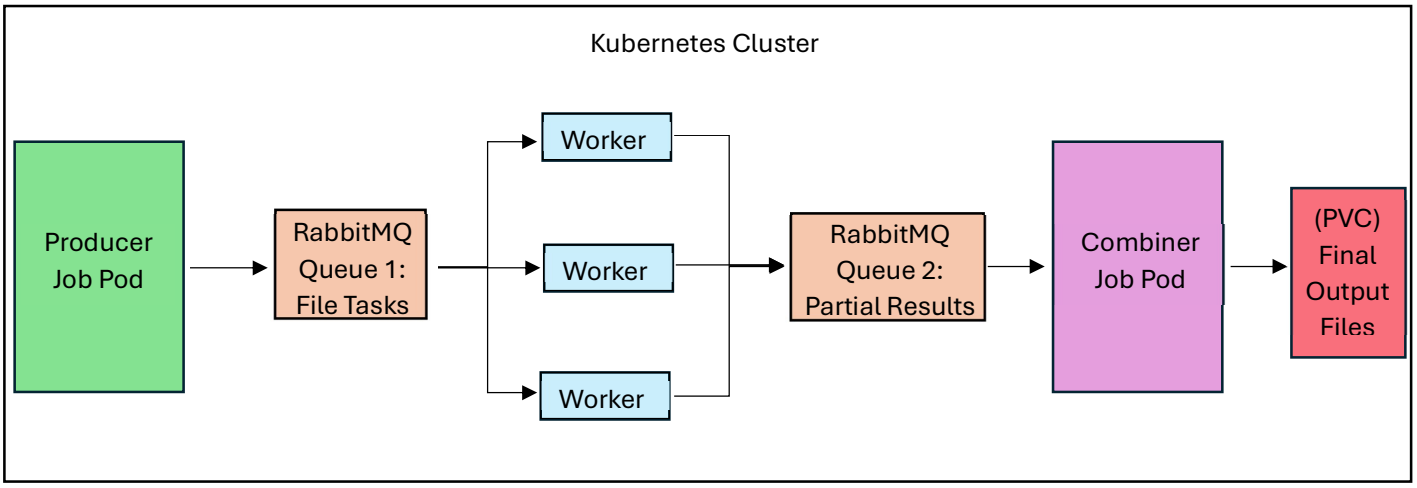

Figure 1: Outline of Kubernetes implementation.

Results & Discussion

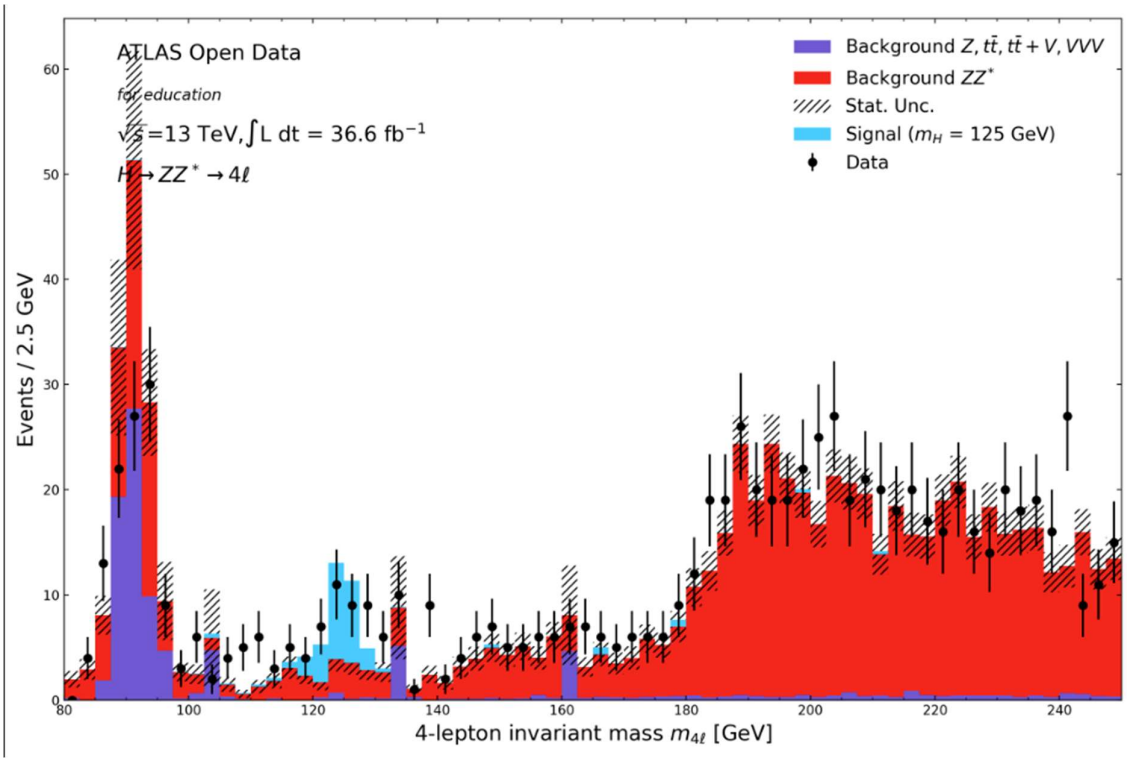

Figure 2: Combined histogram output obtained from Kubernetes run

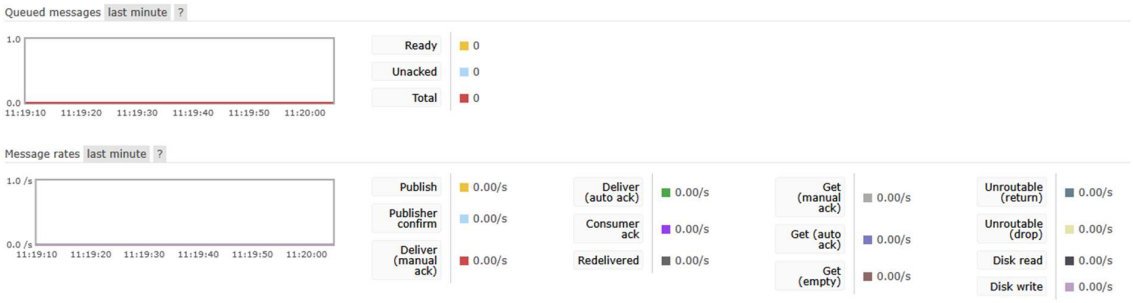

Figure 3: RabbitMQ overview pre Kubernetes job launch

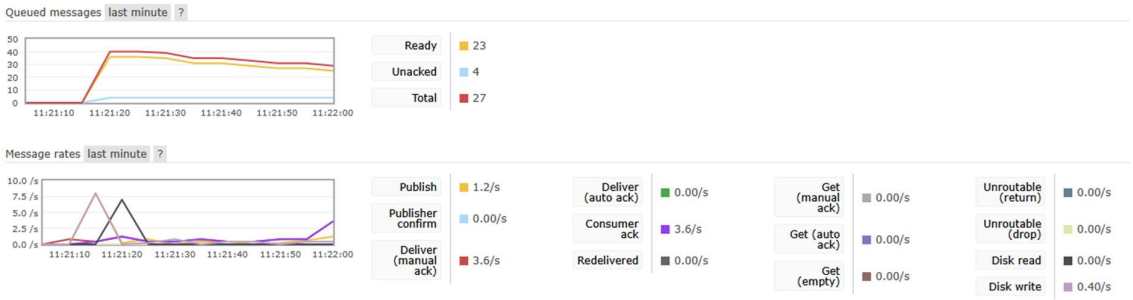

Figure 4: RabbitMQ overview post Kubernetes job launch

Both implementations of Kubernetes and Docker Compose were successful, producing all expected plots and statistics, while reducing total execution time through parallel processing. Final outputs consisted of a png image of Figure 2, a JSON file containing the statistics and a NumPy archive containing processed numerical data. This validated that parallel execution did not alter the scientific result when compared with the original serial workflow.

With the Docker Compose setup, the producer, worker and consumer services all ran inside isolated containers and communicated through RabbitMQ, which was confirmed through observations of the RabbitMQ live overview, showing that messages were published, queued and consumed accordingly. This confirmed that task distribution occurred dynamically through the broker rather than being statically assigned.

With Kubernetes, the producer and combiner were successfully configured as jobs and the workers were deployed as a scalable deployment, completing without failure and producing the expected results. Figures 3 and 4 show a clear change in message state following job execution, where messages were received and queued, with the set number of workers (4) working on a filename at the same time, seen in Figure 4 as the number of messages ‘Unacked’ in the upper graph, with the total number of messages being reduced through execution.

The overall design followed a producer–consumer pattern with a fan-out and fan-in execution model, which reduced coupling between components and allowed failures to be isolated. This structure reduced coupling between components and allowed failures to be isolated without collapsing the entire system. As messages remained in the queue until acknowledgement, worker failure did not result in data loss. This queue-based system also simplified the scaling of the project as additional workers could be introduced without having to modify the code logic, with RabbitMQ managing the workload distribution. However, the system remains limited by RabbitMQ throughput, and with the increase of message size and queue depth, the brokering performance would increasingly affect the overall execution time. Docker significantly reduced host-specific dependency problems by standardising runtime environments across services.

Although Docker Compose allowed all services to run on a single machine, and obtaining all expected results, it lacks orchestration features such as built-in scheduling and fault recovery. It may be suitable for development and small deployments, however for large projects it does not provide production-grade availability guarantees, resulting in the choice of Kubernetes for the final deployment.

Kubernetes introduced workload scheduling, service discovery and container abstraction, making the system more resilient to failure and easier to scale. Despite this benefit, Kubernetes introduced a learning overhead, as the complexity of the configuration increased, especially in volume management and job lifecycles.

Increasing the number of worker replicas, possible by changing its number in the k8s_all.yaml file, decreases total execution time, which is vital for if the dataset increased to an incredibly large number, although this is only a benefit until it is constrained by broker throughput, disk I/O bandwidth, or the serial merging stage in the combiner. The combiner stands as a bottleneck in the codes scalability as all results converge into a single process, which whilst suitable for the moderate workload executed in this project, would require a redesign for very large datasets, such as introducing hierarchical merging or multi-stage aggregation. The producer could also potentially be parallelised if there were to be a much larger dataset due to it currently being a single process. Future improvements could include automatic worker scaling based on queue depth, as well as message retry handling through dead-letter queues.

Conclusion

The sequential data analysis of the original code was successfully transformed into a distributed system using cloud-technologies. With the introduction of message based task distribution of RabbitMQ, containerisation with Docker and orchestration using Kubernetes, the code was rebuilt into a scalable, fault-tolerant workflow. Both Kubernetes and Docker Compose deployments produced identical outputs proving no accuracy was lost in the parallelisation, but with Kubernetes in particular providing significant advantages to the workflow management, making it a much more appropriate solution for scalable workloads. Whilst the system still remains limited by the single node combiner stage and performance of the message broker, the setup demonstrates how cloud-based implementation can improve execution whilst retaining all logic.

References

1. M. Barika, S. Garg, A. Y. Zomaya, L. Wang, A. V. Moorsel and R. Ranjan, *ACM Comput. Surv.*, 2019, **52**, Article 95.
2. J. Elmsheuser and D. van der Ster, *Journal of Physics: Conference Series*, 2012, **396**, 032035.
3. https://github.com/atlas-outreach-data-tools/notebooks-collection-opendata/blob/master/13-TeV-examples/uproot_python/HZZAnalysis.ipynb).
4. Y4_Assignment2, https://github.com/nickhardman3/Y4_Assignment2).